Functional Differentiation in Policy Implementation Texts: Evidence from China's State Council during the 14th Five Year Plan

Heyang Liu

Repository:
https://github.com/heyang-2002/China-Policy-ML

1. Introduction

Governments produce large volumes of policy documents that address multiple governance objectives simultaneously. Measuring how policy attention is distributed across governance domains is therefore difficult using manual reading alone.

This project applies machine learning to classify paragraphs from Chinese State Council policy documents into governance domains. The goal is not merely to demonstrate that machine learning can classify text, but to construct a scalable measurement of policy attention across functional domains.

The central research question is:

Can paragraph-level machine learning classification recover meaningful governance domains from Chinese State Council policy documents?

This question involves several empirical tasks:

1. Construct a large paragraph-level dataset from policy documents.
2. Generate text and structured features suitable for machine learning models.
3. Train supervised classifiers using manually labeled paragraphs.
4. Evaluate model performance using cross-validation.
5. Apply the trained model to the full corpus to measure policy attention.

The following sections describe the complete machine learning pipeline used to answer this question.

2.Data

2.1 Data Source

The data come from the official State Council policy document repository and related government search endpoints. The project focuses on policy documents issued by the State Council, the General Office of the State Council, and affiliated ministries during the 14th Five-Year Plan period (2021–2025).

The corpus includes document-level metadata such as:

- title
- publication date
- issuing organization
- topic classification
- URL
- document type

and full text extracted from document pages.

2.2 Data Access

The first stage of the pipeline retrieves document metadata from the government API. Each request returns a JSON object containing document information.

Then, the API is queried across multiple thematic categories. For each category, the script iterates across result pages and extracts the relevant fields.

The retrieved metadata include:

- title
- publication date
- issuing organization
- topic classification
- document URL

These metadata are stored as a document-level index.

A parted version of the indexing logic is shown below:

```
install.packages(c("httr2", "jsonlite", "dplyr", "purrr", "stringr", "tibble", "readr"))
library(httr2)
library(jsonlite)
library(dplyr)
library(purrr)
library(stringr)
library(tibble)
library(readr)

`%||%` <- function(x, y) if (is.null(x) || length(x) == 0) y else x

fetch_json <- function(childtype, p = 1, n = 50,
                       t_key = "zhengcelibrary_gw_bm",
                       type_key = "gwyzcwjk",
                       mintime = "", maxtime = "") {

  api_url <- "https://sousuo.www.gov.cn/search-gov/data"

  resp <- request(api_url) %>%
    req_url_query(
      t = t_key,
      q = "",
      timetype = "",
      mintime = mintime,
      maxtime = maxtime,
      sort = "score",
      sortType = "1",
      searchfield = "title",
      childtype = as.character(childtype),
      puborg = "",
      pcodeYear = "",
      pcodeNum = "",
      filetype = "",
      p = as.character(p),
      n = as.character(n),
      inpro = "",
      dup = "",
      orpro = "",
      type = type_key
```

```r
    ) %>%
    req_user_agent("Mozilla/5.0") %>%
    req_headers(
      "Accept"="application/json, text/javascript, */*; q=0.01",
      "X-Requested-With"="XMLHttpRequest",
      "Referer"="https://sousuo.www.gov.cn/zcwjk/policyDocumentLibrary"
    ) %>%
    req_perform()

  txt <- resp_body_string(resp)
  fromJSON(txt, simplifyVector = FALSE)
}

extract_cat <- function(json, catKey, childtype, page) {
  rows <- json$searchVO$catMap[[catKey]]$listVO
  if (is.null(rows) || length(rows) == 0) return(tibble())

  tibble(
    theme_childtype = as.character(childtype),
    page = page,
    source_catKey = catKey,                          # gongwen / bumenfile
    source_group = if_else(catKey == "gongwen", "  / ", "   "),
    title = map_chr(rows, ~ .x$title %||% NA_character_),
    pubtimeStr = map_chr(rows, ~ .x$pubtimeStr %||% NA_character_),
    puborg = map_chr(rows, ~ .x$puborg %||% NA_character_),
    pcode = map_chr(rows, ~ .x$pcode %||% NA_character_),
    childtype_text = map_chr(rows, ~ .x$childtype %||% NA_character_),
    url = map_chr(rows, ~ .x$url %||% NA_character_),
    id = map_chr(rows, ~ as.character(.x$id) %||% NA_character_)
  )
}


collect_both <- function(childtype,
                         catKeys = c("gongwen", "bumenfile"),
                         page_size = 50,
                         mintime = "", maxtime = "",
                         t_key = "zhengcelibrary_gw_bm",
                         type_key = "gwyzcwjk",
                         polite_sleep = 0.2) {

  j1 <- fetch_json(childtype = childtype, p = 1, n = page_size,
```

```
                    t_key = t_key, type_key = type_key,
                    mintime = mintime, maxtime = maxtime)

  totals <- map_int(catKeys, ~ j1$searchVO$catMap[[.x]]$totalCount %||% 0)
  names(totals) <- catKeys

  out <- map_dfr(catKeys, function(catKey) {
    total <- totals[[catKey]]
    if (total == 0) return(tibble())

    pages <- ceiling(total / page_size)

    map_dfr(1:pages, function(p) {
      message("childtype=", childtype,
              " catKey=", catKey,
              " page=", p, "/", pages,
              " rows=", page_size)

      j <- if (p == 1) j1 else fetch_json(childtype = childtype, p = p, n = page_size,
                                          t_key = t_key, type_key = type_key,
                                          mintime = mintime, maxtime = maxtime)

      Sys.sleep(polite_sleep)
      extract_cat(j, catKey = catKey, childtype = childtype, page = p)
    })
  })

  out %>%
    distinct(source_catKey, id, url, .keep_all = TRUE)
}
```

2.3 Unit of Analysis

The final unit of analysis is the paragraph rather than the document.

This choice is theoretically and empirically motivated. Document-level categories often reflect administrative filing rather than actual governance function. For example, a document classified under agriculture may contain passages about environmental protection, industrial development, social assistance, or regulation. Paragraph-level segmentation allows the model to capture more fine-grained functional content.

After cleaning and segmentation, the final corpus contains approximately 130,000 paragraphs.

The following code demonstrates how to extract text-level data:

```
library(readr)
library(dplyr)
library(purrr)
library(progress)

# 1) Read Data
idx_all <- read_csv("themes_1080_1099_2021_2025.csv", show_col_types = FALSE)
urls <- idx_all$url %>% unique() %>% na.omit()

# 2) Setwd
out_csv <- "gov_text_extracted_partial.csv"
out_rds <- "gov_text_extracted_partial.rds"

# 3) If you have already run part of the process: Read the completed results, set a breakpoi
done <- if (file.exists(out_csv)) read_csv(out_csv, show_col_types = FALSE) else tibble(url =
done_urls <- done$url %>% unique()

todo_urls <- setdiff(urls, done_urls)

cat("Total:", length(urls), "\n")
cat("Done :", length(done_urls), "\n")
cat("Todo :", length(todo_urls), "\n")

# 4) Progress bar
pb <- progress_bar$new(
  format = "  downloading [:bar] :current/:total (:percent) eta: :eta",
  total = length(todo_urls),
  clear = FALSE,
  width = 60
)

# 5) Capture in batches
batch_size <- 100
batches <- split(todo_urls, ceiling(seq_along(todo_urls) / batch_size))

for (i in seq_along(batches)) {
  message("\nBatch ", i, " / ", length(batches), " ...")
```

```r
out_i <- map_dfr(batches[[i]], function(u) {
  pb$tick()

  # Limit speed
  Sys.sleep(runif(1, 0.2, 0.6))

  fetch_one_doc(u)  # <-
})

# write CSV
if (!file.exists(out_csv)) {
  write_csv(out_i, out_csv)
} else {
  write.table(out_i, out_csv, sep = ",", row.names = FALSE, col.names = FALSE,
              append = TRUE, qmethod = "double", fileEncoding = "UTF-8")
}
```

3. Data Preparation and Feature Engineering

3.1 Document-Level Cleaning

The raw metadata and full text were merged by URL. URLs were standardized, duplicates removed, and document-level variables harmonized. Topic hierarchy variables were generated from existing category labels:

- `topic_lvl1`
- `topic_lvl2`

Document type was inferred from the document title using regular-expression rules that classified titles into categories such as:

- notice
- opinion
- regulation
- plan
- guideline
- scheme
- decision/order

- announcement

3.2 Paragraph Segmentation

Full texts were split into paragraph units based on line breaks. Short or empty segments were removed, and each paragraph received a unique identifier.

The resulting paragraph-level dataset includes:

- paragraph_id
- url
- para_id
- pubdate
- doc_type
- topic_lvl1
- topic_lvl2
- para_text
- para_len

The following code demonstrates how to clean data:

```
text <- read_csv("gov_text_extracted_full.csv", show_col_types = FALSE) %>%
  mutate(url = clean_url(url)) %>%
  distinct(url, .keep_all = TRUE)

merged <- meta %>%
  left_join(text, by = "url")

# Optimize structure
merged <- merged %>%
  mutate(
    # Separate Lv1 topic and LV2 topic
    topic_lvl1 = str_trim(str_extract(childtype_text, "^[^\\\\]+")),
    topic_lvl2 = str_trim(str_replace(childtype_text, "^[^\\\\]+\\\\?", "")),
    topic_lvl2 = if_else(topic_lvl2 == "" | is.na(topic_lvl2), NA_character_, topic_lvl2),

    # Mark the main text
    has_text = !is.na(text) & nchar(text) > 200,
```

```r
    # Text length
    text_len = nchar(text)
  )

# Delete useless lists
analysis_df <- merged %>%
  select(
    url,
    title,
    pubdate,
    source_group,
    puborg,
    pcode,
    doc_type,
    topic_lvl1,
    topic_lvl2,
    text,
    text_len,
    has_text,
    attachments
  )
write_csv(analysis_df,"analysis1.csv")


# Segmentation processing
library(tidyverse)

split_paragraphs <- function(text) {
  if (is.na(text) || str_trim(text) == "") return(tibble())

  tibble(para_text = str_split(text, "\\n+")[[1]]) %>%
    mutate(para_text = str_trim(para_text)) %>%
    filter(nchar(para_text) >= 20) %>%     #   /
    mutate(
      para_len = nchar(para_text),
      para_id = row_number()
    )
}

para_df <- analysis_df %>%
  select(url, doc_type, topic_lvl1, topic_lvl2, pubdate, text) %>%
  mutate(paragraphs = map(text, split_paragraphs)) %>%
```

```
  select(-text) %>%
  unnest(paragraphs)

glimpse(para_df)
```

4.Exploratory Data Analysis

Before training machine learning models, I conducted exploratory data analysis (EDA) to understand the structure of the paragraph-level dataset. EDA serves several purposes in this project:

1. to examine the distribution of document types and topics,
2. to understand the statistical properties of paragraph text,
3. to identify potential feature imbalances that may affect model training,
4. to validate the paragraph-level representation used for classification.

The analyses below summarize the main structural properties of the dataset.

4.1 Document Type Distribution

Policy documents in the corpus include multiple document types such as notices, opinions, plans, regulations, and implementation guidelines. These types differ in both administrative function and writing style, which may affect how policy content is expressed in text.

To examine the distribution of document types, I first computed paragraph counts by document type.

```
setwd("~/Desktop/chineseresearch")
para_df <- read.csv("para_analysis.csv")
library(tidyverse)
```

```
Warning: package 'ggplot2' was built under R version 4.4.3

Warning: package 'tibble' was built under R version 4.4.3

Warning: package 'readr' was built under R version 4.4.3

Warning: package 'purrr' was built under R version 4.4.3
```

```
Warning: package 'dplyr' was built under R version 4.4.3

-- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
v dplyr     1.2.0     v readr     2.2.0
v forcats   1.0.0     v stringr   1.6.0
v ggplot2   4.0.2     v tibble    3.3.1
v lubridate 1.9.4     v tidyr     1.3.1
v purrr     1.2.1
-- Conflicts ------------------------------------------ tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()    masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to becom
```

```r
para_df %>%
  count(doc_type) %>%
  arrange(desc(n))
```

```
        doc_type     n
1             68244
2             22591
3             17228
4              8349
5              7051
6       /   6460
7       /   3145
8              1355
9               377
10               213
11    /     143
12   -    128
13           119
14   -     85
15               42
16               33
17               32
18               29
19   -     24
20   -     21
21      /     10
```

This distribution provides a basic view of how different policy formats contribute to the overall

dataset. Some document types appear far more frequently than others, indicating that the corpus is dominated by a small number of administrative formats.

Because these formats often correspond to different policy instruments, the document type variable is later included as a structured feature in the machine learning models.

To visualize the distribution, I generated the following bar chart:

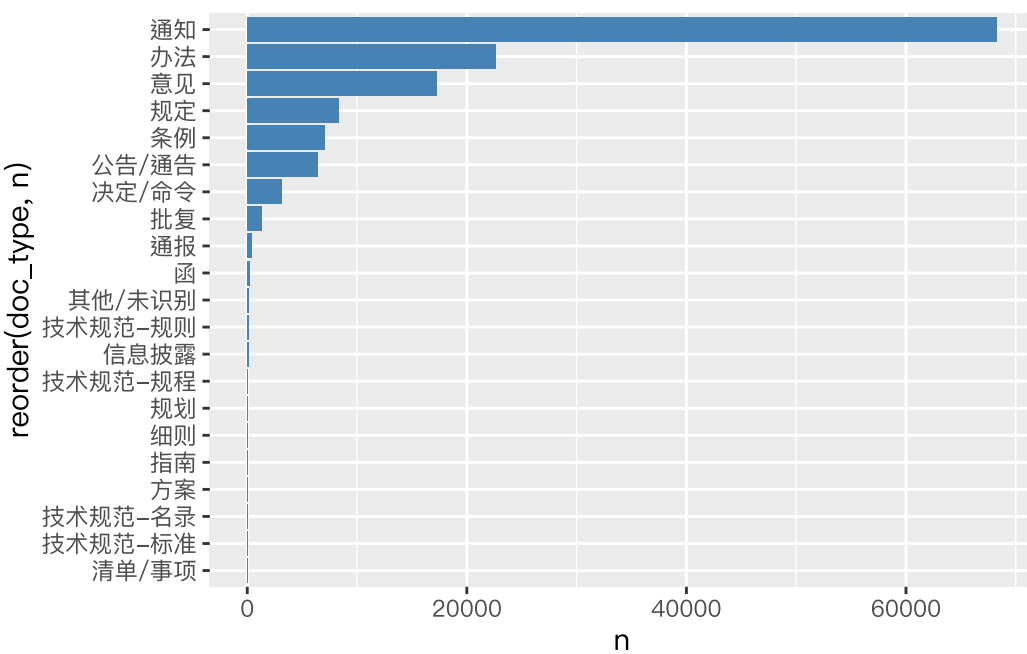
```
library(showtext)
```

```
Loading required package: sysfonts
```

```
Loading required package: showtextdb
```

```
font_add("cn", "PingFang.ttc")
showtext_auto()

para_df %>%
  count(doc_type, sort = TRUE) %>%
  ggplot(aes(x = reorder(doc_type, n), y = n)) +
  geom_col(fill = "steelblue") +
  theme(text = element_text(family = "cn")) +
  coord_flip()
```

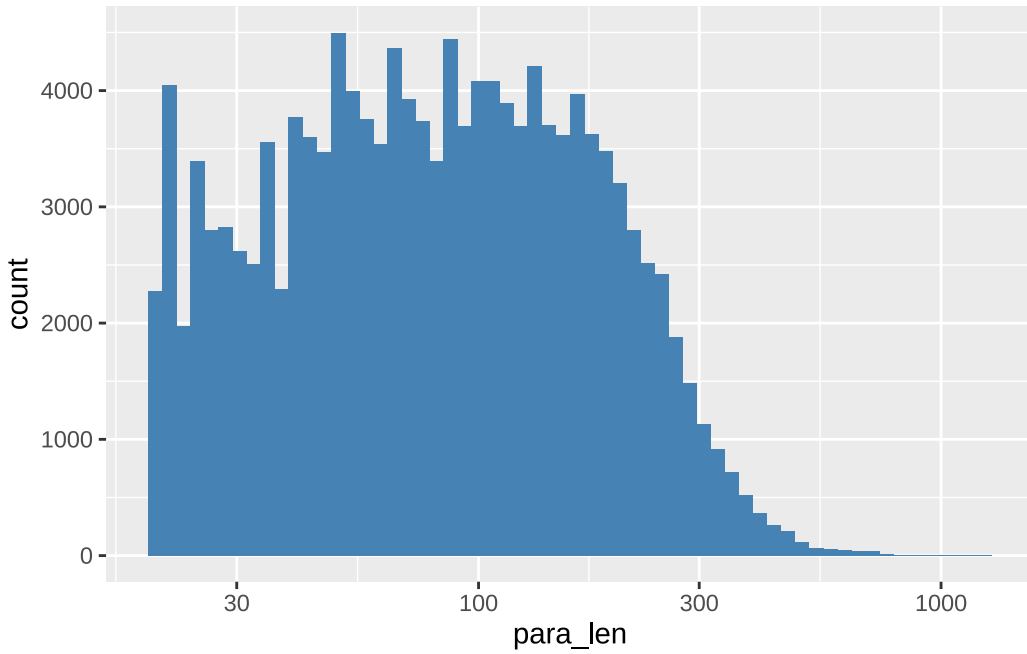


4.2 Paragraph Length Distribution

Paragraph length is an important characteristic of the text data. Extremely short paragraphs may contain insufficient semantic information for reliable classification, while very long paragraphs may contain multiple policy objectives.

To examine paragraph length distribution, I plotted a histogram of paragraph length.

```
ggplot(para_df, aes(x = para_len)) +
  geom_histogram(bins = 60, fill = "steelblue") +
  scale_x_log10()
```



The resulting distribution is strongly right-skewed, indicating that most paragraphs are relatively short while a smaller number of paragraphs are substantially longer. This pattern is common in policy texts, where short administrative directives coexist with longer explanatory paragraphs.

Because the distribution is highly skewed, a logarithmic scale is used to improve visualization and highlight variation among shorter paragraphs.

This finding has implications for feature engineering. Paragraph length may serve as a proxy for textual information density, and therefore can be incorporated into attention-weighted metrics in later analysis. However, extremely long paragraphs may disproportionately influence aggregate measures. To address this issue, a capped paragraph length metric is later introduced when constructing attention indicators.

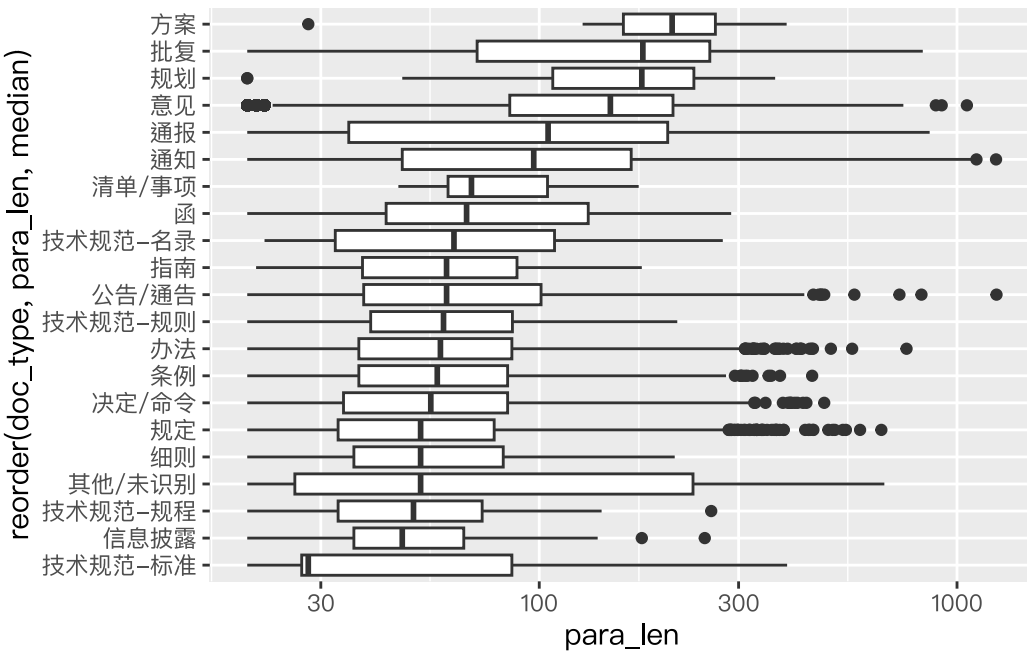
4.3 Paragraph Length by Document Type

Different document types may exhibit systematic differences in writing style. For example, formal regulatory texts may use longer and more structured paragraphs, whereas administrative notices may rely on shorter directives.

To investigate this possibility, I examined paragraph length distributions across document types.

```
library(showtext)
font_add("cn", "PingFang.ttc")
showtext_auto()

ggplot(para_df, aes(x = reorder(doc_type, para_len, median),
                    y = para_len)) +
  geom_boxplot() +
  scale_y_log10() +
  theme(text = element_text(family = "cn")) +
  coord_flip()
```



The results reveal substantial variation in paragraph length across document types. Some document formats tend to produce longer paragraphs, while others consist primarily of shorter directives.

This variation suggests that document type is not merely a descriptive variable but may contain informative structural signals relevant to text classification. For example, regulatory documents often contain more detailed explanatory text, whereas administrative notices may rely on shorter instructions.

Consequently, document type is included as a structured feature in the machine learning models.

4.4 Topic Distribution

The policy documents are categorized according to a hierarchical topic classification system. The first-level topic categories correspond to broad policy domains, while the second-level topics provide more detailed thematic classifications.

To examine the distribution of topics in the dataset, I computed paragraph counts by topic.

```
para_df %>%
  count(topic_lvl1, sort = TRUE)
```

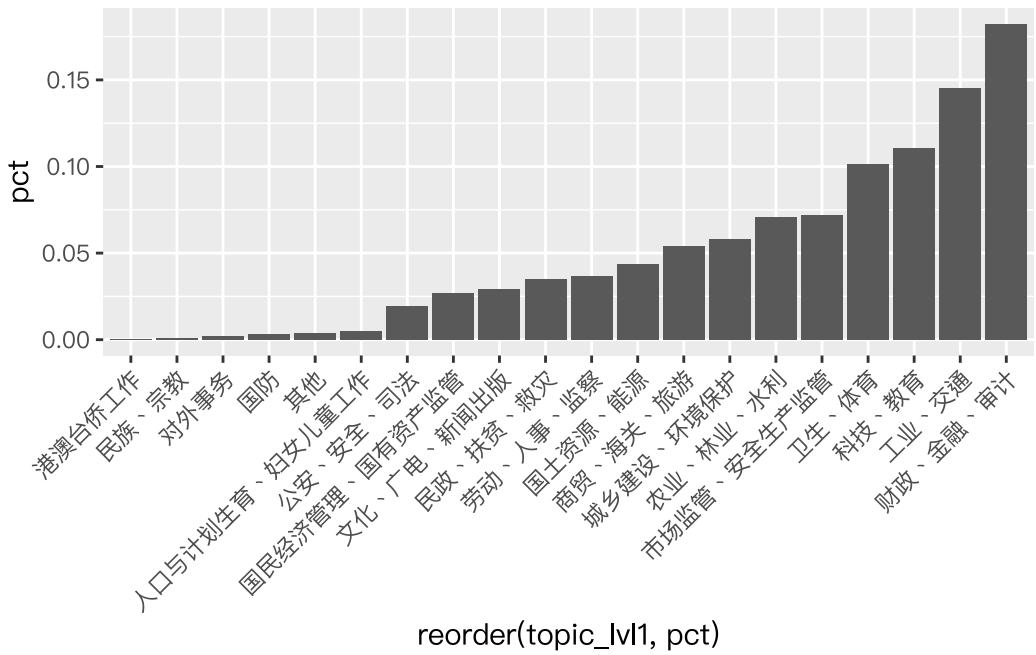
```
                     topic_lvl1     n
1                    24749
2                        19676
3                        15013
4                        13761
5                 9753
6                     9583
7                    7856
8                     7356
9                       5922
10                    5012
11                    4715
12                  3940
13             3618
14                    2622
15            645
16                            495
17                            459
18                          306
19                         120
20                         78
```

The results indicate that the corpus covers a wide range of policy areas, including economic management, environmental governance, social policy, and public safety.

To better visualize the relative share of each topic category, I constructed the following plot.

```
library(showtext)
font_add("cn", "PingFang.ttc")
showtext_auto()

para_df %>%
  count(topic_lvl1) %>%
  mutate(pct = n / sum(n)) %>%
  ggplot(aes(x = reorder(topic_lvl1, pct), y = pct)) +
  theme(text = element_text(family = "cn")) +
  theme(axis.text.x = element_text(angle = 45, hjust = 1)) +
  geom_col()
```



The resulting distribution shows that policy topics are **unevenly distributed across the dataset**. A small number of policy domains account for a large share of paragraphs, while other topics appear much less frequently.

This imbalance is important for machine learning classification. Highly imbalanced class distributions can bias model training, leading classifiers to favor majority categories. As a result,

evaluation metrics beyond overall accuracy—such as precision, recall, and F1-score—are necessary to properly assess model performance.

Furthermore, class imbalance may motivate the use of techniques such as class weighting or stratified sampling during model training.

4.5 Temporal Distribution

Policy documents in the corpus span multiple years. Examining temporal patterns helps determine whether the dataset is evenly distributed over time or dominated by particular periods.

The following code computes the number of paragraphs by year and document type.

```
para_df %>%
  mutate(year = lubridate::year(pubdate)) %>%
  count(year, doc_type)
```

```
   year      doc_type     n
1  2021      /   1557
2  2021    /      58
3  2021      /    387
4  2021                103
5  2021             7930
6  2021             5158
7  2021              163
8  2021   -      6
9  2021   -      6
10 2021   -    128
11 2021             1863
12 2021             3636
13 2021              161
14 2021            18083
15 2022           116
16 2022      /   1156
17 2022    /      11
18 2022      /   1877
19 2022                  52
20 2022             5458
21 2022             3400
22 2022              226
23 2022   -     15
```

24 2022 - 85
25 2022 378
26 2022 2132
27 2022 75
28 2022 15158
29 2023 / 1649
30 2023 / 46
31 2023 / 633
32 2023 5
33 2023 4000
34 2023 2702
35 2023 240
36 2023 - 3
37 2023 32
38 2023 29
39 2023 1145
40 2023 / 10
41 2023 33
42 2023 42
43 2023 853
44 2023 55
45 2023 13314
46 2024 3
47 2024 / 1128
48 2024 / 23
49 2024 / 248
50 2024 21
51 2024 3211
52 2024 3332
53 2024 310
54 2024 2009
55 2024 797
56 2024 43
57 2024 13353
58 2025 / 970
59 2025 / 5
60 2025 32
61 2025 1992
62 2025 2636
63 2025 416
64 2025 - 15
65 2025 1656
66 2025 931

```
67 2025                43
68 2025              8336
```

The results show that paragraph counts vary across years and document types. This variation reflects the evolving policy agenda as well as changes in administrative output across the study period.

Temporal variation is important for later analysis because it allows the machine learning predictions to be used for studying policy attention dynamics over time.

4.6 Consistency of Attention Measures

The final exploratory analysis evaluates the consistency of different measures of policy attention.

Three potential measures were considered:

- number of paragraphs
- total paragraph length
- capped paragraph length

The following code computes these measures at the document level.

```r
att_test <- para_df %>%
  group_by(url) %>%
  summarise(
    n_paras = n(),
    total_len = sum(para_len),
    capped_len = sum(pmin(para_len, 300))
  )

cor(att_test[, c("n_paras", "total_len", "capped_len")], use = "complete.obs")
```

```
             n_paras total_len capped_len
n_paras    1.0000000 0.8611351  0.8754104
total_len  0.8611351 1.0000000  0.9980086
capped_len 0.8754104 0.9980086  1.0000000
```

The results indicate strong correlations among the three indicators. In particular, the correlation between total length and capped length is extremely high (0.998), suggesting that truncating paragraph length has minimal impact on the overall attention measure. This implies that extremely long paragraphs are relatively rare in the dataset and do not substantially distort the distribution of textual attention.

The correlation between **paragraph counts** and **length-based measures** is slightly lower (around 0.86–0.88), indicating that the number of paragraphs and the total textual volume capture related but not identical aspects of policy attention. Paragraph counts emphasize the frequency of policy statements, while length-based measures capture the amount of textual space devoted to policy content.

Because these measures capture slightly different dimensions of policy attention, both paragraph-based and length-based indicators are retained in subsequent analyses.

5.Construction of the Labeled Training Sample

To train supervised machine learning models for policy domain classification, a labeled training dataset was constructed from the paragraph-level corpus. Because the full dataset contains a very large number of paragraphs, manual labeling of the entire corpus is infeasible. Instead, a structured sampling strategy was used to generate a representative subset of paragraphs for annotation.

The sampling procedure aims to satisfy three objectives:

maintain temporal coverage across years,

ensure semantic diversity across policy topics,

control for the presence of administrative boilerplate text.

5.1 Text Cleaning and Preprocessing

Before sampling, the dataset was cleaned to remove empty or extremely short paragraphs. Paragraph text was standardized and basic metadata variables were constructed.

The following preprocessing steps were applied:

convert the year variable to integer format,

remove leading and trailing whitespace from text fields,

compute paragraph length,

remove paragraphs shorter than 30 characters,

ensure each paragraph appears only once.

An administrative boilerplate score was also constructed based on the frequency of administrative phrases within the paragraph text. This score is normalized by paragraph length to account for variation in text size.

```
df2 <- df %>%
  mutate(
    year = as.integer(year),
    topic_lvl1 = str_squish(as.character(topic_lvl1)),
    paragraph_text = str_squish(as.character(paragraph_text)),
    text_len = nchar(paragraph_text)
  ) %>%
  filter(
    !is.na(year),
    !is.na(topic_lvl1), topic_lvl1 != "",
    !is.na(paragraph_text), paragraph_text != "",
    text_len >= 30
  ) %>%
  distinct(paragraph_id, .keep_all = TRUE) %>%
  mutate(
    admin_hits  = str_count(paragraph_text, admin_pattern),
    admin_score = admin_hits / pmax(text_len, 1)
  )
```

5.2 Constructing Clean and Boilerplate Pools

Policy documents often contain a substantial amount of administrative boilerplate language, such as formal procedural phrases. If the training data were sampled randomly from the entire corpus, the labeled dataset might be dominated by these administrative expressions rather than substantive policy content.

To address this issue, the dataset was divided into two pools based on the administrative score:

clean pool: paragraphs with relatively low administrative content

boilerplate pool: paragraphs with high administrative phrase density

The threshold separating these two pools is determined using a quantile of the administrative score distribution.

```
cut <- quantile(df2$admin_score, probs = admin_quantile_keep, na.rm = TRUE)

pool_clean  <- df2 %>% filter(admin_score <= cut)
pool_boiler <- df2 %>% filter(admin_score >  cut)
```

5.3 Topic-Based Sampling Using LDA

To increase semantic coverage in the labeled dataset, topic modeling was used as a semantic anchor during the sampling process.

Specifically, a Latent Dirichlet Allocation (LDA) model was applied to the paragraph texts to identify latent semantic topics. Each paragraph was then assigned a dominant topic based on the highest posterior probability.

The following code illustrates the topic modeling step.

```
lda <- LDA(dtm, k = k_topics, control = list(seed = seed))
gamma <- posterior(lda)$topics
dominant <- apply(gamma, 1, which.max)
```

5.4 Stratified Sampling Across Years and Topics

The final sampling procedure is conducted in two stages:

Year-level allocation Paragraphs are first distributed proportionally across years to maintain temporal coverage.

Topic-level sampling within each year Within each year, paragraphs are sampled across dominant topics to ensure semantic diversity.

This stratified sampling strategy reduces the risk that the labeled dataset is dominated by a small number of policy topics or time periods.

5.5 Final Labeled Dataset

After sampling from both the clean and boilerplate pools, the sampled paragraphs are combined to form the final labeling dataset.

The final output is a labeling sheet containing the sampled paragraphs and associated metadata. This sheet includes empty columns for manual annotation.

```
label_sheet <- sample_df %>%
  transmute(
    paragraph_id,
    url,
    para_id,
    year,
    doc_type,
    topic_lvl1,
```

```
    topic_lvl2,
    pool_tag,
    dominant_topic,
    admin_score,
    paragraph_text,
    domain_label = "",
    domain_secondary = "",
    instrument_labels = ""
  )
```

Annotators are asked to assign labels to the following fields:

- domain_label: primary policy domain
- domain_secondary: secondary domain if applicable
- instrument_labels: policy instrument categories

This labeled dataset serves as the training data for the supervised machine learning models described in the next section.

6. Method and Model Implementation

To classify policy implementation paragraphs into governance domains, this study constructs a supervised machine learning pipeline combining textual features and structured policy information. The workflow follows four main steps: data preparation, feature construction, model training, and performance evaluation.

6.1 Labeled Data

After constructing the labeled sample, the next step is to train and compare supervised classification models. The training pipeline starts from a manually labeled dataset of 500 paragraphs, where each observation contains both textual content and structured metadata. The primary label is stored in `domain_label`, while `domain_secondary` is optionally recorded for paragraphs that plausibly belong to more than one policy domain. This secondary label is not used for training the classifier directly, but it is retained for a more flexible evaluation later.

```
setwd("~/Desktop/chineseresearch")
label <- read.csv("label.csv")
library(tidyverse)
library(tidymodels)
```

```
-- Attaching packages ------------------------------------- tidymodels 1.4.1 --

v broom        1.0.12     v rsample      1.3.2
v dials        1.4.2      v tailor       0.1.0
v infer        1.1.0      v tune         2.0.1
v modeldata    1.5.1      v workflows    1.3.0
v parsnip      1.4.1      v workflowsets 1.1.1
v recipes      1.3.1      v yardstick    1.3.2

Warning: package 'broom' was built under R version 4.4.3

Warning: package 'infer' was built under R version 4.4.3

Warning: package 'parsnip' was built under R version 4.4.3

Warning: package 'rsample' was built under R version 4.4.3

-- Conflicts ---------------------------------------- tidymodels_conflicts() --
x scales::discard() masks purrr::discard()
x dplyr::filter()   masks stats::filter()
x recipes::fixed()  masks stringr::fixed()
x dplyr::lag()      masks stats::lag()
x yardstick::spec() masks readr::spec()
x recipes::step()   masks stats::step()
```

```r
library(tokenizers)
library(purrr)
library(textrecipes)


label500 <- label %>%
  filter(domain_label != "") %>%
  transmute(
    text = paragraph_text,
    domain = factor(domain_label),
    domain_secondary = na_if(domain_secondary, ""),
    doc_type = factor(doc_type),
    topic_lvl1 = factor(topic_lvl1),
    topic_lvl2 = factor(topic_lvl2),
    dominant_topic = factor(dominant_topic),
```

```
    admin_score = as.numeric(admin_score)
  ) %>%
  mutate(
    text_bigram = map_chr(
      text,
      ~ {
        bg <- tokenizers::tokenize_ngrams(.x, n = 2, n_min = 2)[[1]]
        if (length(bg) == 0) "" else paste(bg, collapse = " ")
      }
    )
  )
```

This step does several things at once. First, it restricts the dataset to observations that have valid primary labels, so the final training set contains only manually reviewed examples. Second, it keeps both raw text and structured predictors, including document type, topic hierarchy, dominant topic category, and the administrative score. Third, it creates a bigram representation of the text. I include bigrams because many policy-relevant concepts appear in short phrases rather than as isolated words. In this project, the classifier is therefore trained not only on single-word TF-IDF features, but also on phrase-level information extracted from the same paragraph.

Once the labeled dataset is prepared, I split it into a training set and a test set.

```
set.seed(123)
split <- initial_split(label500, prop = 0.8, strata = domain)
train <- training(split) %>% mutate(row_id = row_number())
test  <- testing(split)  %>% mutate(row_id = row_number())
```

The training set contains 80% of the labeled paragraphs, while the remaining 20% are held out for final testing. The split is stratified by `domain`, which means the class distribution is preserved across the training and testing subsets. This matters because policy domains are not perfectly balanced; without stratification, some categories might become underrepresented in either the training set or the test set. I also add a `row_id` variable so that predictions can later be matched back to the corresponding observations when merging secondary labels into the evaluation output.

6.2 Cross-validation

Within the training set, five-fold cross-validation is implemented to ensure robust performance estimation during model development.

This means that the training data are repeatedly divided into five folds, and each fold is used once as a validation set while the remaining four folds are used for model fitting. Cross-validation is especially important here because the labeled sample is relatively small. Instead of trusting performance estimates from a single partition, the cross-validation procedure provides a more stable assessment of how the model behaves across different subsets of the training data.

```
set.seed(123)
folds <- vfold_cv(train, v = 5, strata = domain)
```

6.3 Feature Engineering

Feature engineering is implemented using the tidymodels recipe framework, which constructs a unified preprocessing pipeline. The text is first tokenized and filtered to retain the most informative vocabulary items. The resulting tokens are then transformed into TF-IDF representations, which weight words according to their importance in distinguishing documents.

```
rec <- recipe(
  domain ~ text + text_bigram + doc_type + topic_lvl1 + topic_lvl2 + dominant_topic + admin_s
  data = train
) %>%
  step_tokenize(text) %>%
  step_tokenfilter(text, max_tokens = 8000) %>%
  step_tfidf(text) %>%
  step_tokenize(text_bigram) %>%
  step_tokenfilter(text_bigram, max_tokens = 8000) %>%
  step_tfidf(text_bigram) %>%
  step_dummy(all_nominal_predictors()) %>%
  step_normalize(admin_score)
```

The TF-IDF transformation converts textual data into a high-dimensional numeric representation that reflects the relative importance of words or phrases across policy paragraphs. Structured variables are converted into dummy variables, and the administrative score is normalized to ensure comparability with other predictors.

6.4 LASSO

Two classification models are trained and compared: multinomial logistic regression with LASSO regularization and a linear support vector machine. Both models are commonly used for high-dimensional text classification problems.

The multinomial logistic regression model is implemented using the glmnet engine. This model treats the classification problem as a multinomial extension of logistic regression, while the L1 penalty shrinks less useful coefficients toward zero. In a text classification problem, this is valuable because the number of predictors can become very large once TF-IDF features are included. Regularization helps the model avoid fitting noise in the training data.

Because the strength of regularization matters, I tune the penalty parameter rather than fixing it arbitrarily.

The tuning step evaluates the model across a regular grid of 20 candidate penalty values. For each value, the model is trained and validated across the five cross-validation folds. I evaluate each setting using both F1 score and accuracy, and I save the fold-level predictions so that I can later compute additional metrics and inspect confusion patterns.

```r
glmnet_mod <- multinom_reg(penalty = tune(), mixture = 1) %>%
  set_engine("glmnet")

wf_glmnet <- workflow() %>%
  add_recipe(rec) %>%
  add_model(glmnet_mod)

grid <- grid_regular(penalty(), levels = 20)

set.seed(123)
res_glmnet <- tune_grid(
  wf_glmnet,
  resamples = folds,
  grid = grid,
  metrics = metric_set(f_meas, accuracy),
  control = control_grid(save_pred = TRUE)
)
```

```
> A | warning: max_tokens was set to 8000, but only 3369 was available and selected.

There were issues with some computations   A: x2

> B | warning: While computing multiclass `precision()`, some levels had no predicted events
               (i.e. `true_positive + false_positive = 0`).
               Precision is undefined in this case, and those levels will be removed from the
               averaged result.
               Note that the following number of true events actually occurred for each
               problematic event level:
               '     ': 9
```

```
There were issues with some computations   A: x2
> C | warning: While computing multiclass `precision()`, some levels had no predicted events
               (i.e. `true_positive + false_positive = 0`).
               Precision is undefined in this case, and those levels will be removed from the
               averaged result.
               Note that the following number of true events actually occurred for each
               problematic event level:
               '  ': 13, '     ': 11, '    ': 22,
               '     ': 9
There were issues with some computations   A: x2
There were issues with some computations   A: x4   B: x1   C: x2
> D | warning: max_tokens was set to 8000, but only 3292 was available and selected.
There were issues with some computations   A: x4   B: x1   C: x2
> E | warning: While computing multiclass `precision()`, some levels had no predicted events
               (i.e. `true_positive + false_positive = 0`).
               Precision is undefined in this case, and those levels will be removed from the
               averaged result.
               Note that the following number of true events actually occurred for each
               problematic event level:
               '     ': 3
There were issues with some computations   A: x4   B: x1   C: x2
> F | warning: While computing multiclass `precision()`, some levels had no predicted events
               (i.e. `true_positive + false_positive = 0`).
               Precision is undefined in this case, and those levels will be removed from the
               averaged result.
               Note that the following number of true events actually occurred for each
               problematic event level:
               '  ': 16, '     ': 11, '    ': 21,
               '     ': 3
There were issues with some computations   A: x4   B: x1   C: x2
There were issues with some computations   A: x4   B: x1   C: x2   D: x4   E:...
> G | warning: max_tokens was set to 8000, but only 3368 was available and selected.
There were issues with some computations   A: x4   B: x1   C: x2   D: x4   E:...
There were issues with some computations   A: x4   B: x1   C: x2   D: x4   E:...
> H | warning: While computing multiclass `precision()`, some levels had no predicted events
               (i.e. `true_positive + false_positive = 0`).
               Precision is undefined in this case, and those levels will be removed from the
               averaged result.
               Note that the following number of true events actually occurred for each
               problematic event level:
               '     ': 12, '     ': 4
There were issues with some computations   A: x4   B: x1   C: x2   D: x4   E:...
> I | warning: While computing multiclass `precision()`, some levels had no predicted events
```

```
                (i.e. `true_positive + false_positive = 0`).
                Precision is undefined in this case, and those levels will be removed from the
                averaged result.
                Note that the following number of true events actually occurred for each
                problematic event level:
                '  ': 15, '     ': 12, '    ': 21,
                '     ': 4
There were issues with some computations   A: x4   B: x1   C: x2   D: x4   E:...
There were issues with some computations   A: x4   B: x1   C: x2   D: x4   E:...
> J | warning: max_tokens was set to 8000, but only 3312 was available and selected.
There were issues with some computations   A: x4   B: x1   C: x2   D: x4   E:...
> K | warning: While computing multiclass `precision()`, some levels had no predicted events
                 (i.e. `true_positive + false_positive = 0`).
                 Precision is undefined in this case, and those levels will be removed from th
                 averaged result.
                 Note that the following number of true events actually occurred for each
                 problematic event level:
                 '     ': 6
There were issues with some computations   A: x4   B: x1   C: x2   D: x4   E:...
> L | warning: While computing multiclass `precision()`, some levels had no predicted events
                 (i.e. `true_positive + false_positive = 0`).
                 Precision is undefined in this case, and those levels will be removed from th
                 averaged result.
                 Note that the following number of true events actually occurred for each
                 problematic event level:
                 '  ': 14, '     ': 11, '    ': 22,
                 '     ': 6
There were issues with some computations   A: x4   B: x1   C: x2   D: x4   E:...
There were issues with some computations   A: x4   B: x1   C: x2   D: x4   E:...
> M | warning: max_tokens was set to 8000, but only 3332 was available and selected.
There were issues with some computations   A: x4   B: x1   C: x2   D: x4   E:...
> N | warning: While computing multiclass `precision()`, some levels had no predicted events
                 (i.e. `true_positive + false_positive = 0`).
                 Precision is undefined in this case, and those levels will be removed from th
                 averaged result.
                 Note that the following number of true events actually occurred for each
                 problematic event level:
                 '  ': 13, '     ': 12, '    ': 22,
                 '     ': 3
There were issues with some computations   A: x4   B: x1   C: x2   D: x4   E:...
There were issues with some computations   A: x4   B: x1   C: x2   D: x4   E:...
There were issues with some computations   A: x4   B: x1   C: x2   D: x4   E:...
```

To summarize the model's performance, I first compute the cross-validated macro F1 score.

```
macro_f1_glmnet <- res_glmnet %>%
  collect_predictions() %>%
  f_meas(truth = domain, estimate = .pred_class, estimator = "macro")
```

I use macro F1 rather than only raw accuracy because this is a multiclass problem with some class imbalance. Macro F1 gives equal importance to each class by averaging F1 across domains. This prevents the performance estimate from being dominated by the largest categories.

Next, I select the best-tuned penalty value and use it to construct a confusion matrix from the cross-validation predictions.

```
best_glmnet <- select_best(res_glmnet, metric = "f_meas")

glmnet_pred_cv <- res_glmnet %>%
  collect_predictions(parameters = best_glmnet) %>%
  left_join(
    train %>%
      mutate(.row = row_number()) %>%
      select(.row, domain_secondary),
    by = ".row"
  )

cm_glmnet <- glmnet_pred_cv %>%
  conf_mat(truth = domain, estimate = .pred_class)
```

This part of the code does two important things. First, it isolates the predictions corresponding to the best-performing tuned model. Second, it merges the secondary domain labels back into the prediction table. That merge becomes useful later when I compute relaxed accuracy, since a prediction can be treated as acceptable if it matches either the primary or the secondary label.

6.5 SVM

The second model is a linear support vector machine using `LiblineaR`.

```
svm_mod <- svm_linear(mode = "classification") %>%
  set_engine("LiblineaR")
```

```r
wf_svm <- workflow() %>%
  add_recipe(rec) %>%
  add_model(svm_mod)

set.seed(123)
res_svm <- fit_resamples(
  wf_svm,
  resamples = folds,
  metrics = metric_set(f_meas, accuracy),
  control = control_resamples(save_pred = TRUE)
)
```

```
> A | warning: max_tokens was set to 8000, but only 3369 was available and selected.

> B | warning: max_tokens was set to 8000, but only 3292 was available and selected.

There were issues with some computations   A: x2   B: x2

> C | warning: max_tokens was set to 8000, but only 3368 was available and selected.

There were issues with some computations   A: x2   B: x2
> D | warning: max_tokens was set to 8000, but only 3312 was available and selected.
There were issues with some computations   A: x2   B: x2
> E | warning: max_tokens was set to 8000, but only 3332 was available and selected.
There were issues with some computations   A: x2   B: x2
There were issues with some computations   A: x2   B: x2   C: x2   D: x2   E: x2
There were issues with some computations   A: x2   B: x2   C: x2   D: x2   E: x2
```

The support vector machine uses the exact same feature recipe and the same cross-validation folds as the logistic model. This keeps the comparison fair. The only difference is the classifier itself. In practice, linear SVMs often perform well in sparse high-dimensional text spaces, so it is a natural model to compare against regularized logistic regression.

As with the glmnet model, I summarize the SVM results using macro F1 and a confusion matrix.

```r
macro_f1_svm <- res_svm %>%
  collect_predictions() %>%
  f_meas(truth = domain, estimate = .pred_class, estimator = "macro")
```

```r
svm_pred_cv <- res_svm %>%
  collect_predictions() %>%
  left_join(
    train %>%
      mutate(.row = row_number()) %>%
      select(.row, domain_secondary),
    by = ".row"
  )

cm_svm <- svm_pred_cv %>%
  conf_mat(truth = domain, estimate = .pred_class)
```

At this stage, both models have been trained and evaluated under the same preprocessing and validation setup, which makes it possible to compare them directly.

In addition to standard accuracy and macro F1, I also define a relaxed evaluation metric. This is because some policy paragraphs genuinely contain content from more than one governance domain. The labeling process captures this possibility by allowing a secondary domain label. A prediction is therefore counted as relaxed-correct if it matches either the primary or the secondary label.

```r
relaxed_metrics <- function(pred_df) {
  pred_df %>%
    mutate(
      correct_relaxed =
        (.pred_class == domain) |
        (!is.na(domain_secondary) & .pred_class == domain_secondary)
    ) %>%
    summarise(
      relaxed_accuracy = mean(correct_relaxed),
      n = n()
    )
}

relaxed_glmnet_cv <- relaxed_metrics(glmnet_pred_cv)
relaxed_svm_cv    <- relaxed_metrics(svm_pred_cv)
```

This metric is especially useful in policy text because categorical boundaries are not always sharp. Some paragraphs discuss, for example, both macroeconomic coordination and industrial development, or both social policy and public safety. The relaxed metric does not replace the primary-label evaluation, but it provides a second perspective on model performance in a setting where domain overlap is substantively meaningful.

After cross-validation, I fit the final versions of both models on the full training set and evaluate them on the held-out test set.

For the tuned logistic regression model:

```
final_wf_glmnet <- finalize_workflow(wf_glmnet, best_glmnet)

final_fit_glmnet <- final_wf_glmnet %>% fit(data = train)
```

```
Warning: max_tokens was set to 8000, but only 3708 was available and selected.
max_tokens was set to 8000, but only 3708 was available and selected.
```

```
test_pred_glmnet <- predict(final_fit_glmnet, new_data = test) %>%
  bind_cols(test %>% select(domain, domain_secondary))

test_macro_f1_glmnet <- test_pred_glmnet %>%
  f_meas(truth = domain, estimate = .pred_class, estimator = "macro")
```

```
Warning: While computing multiclass `precision()`, some levels had no predicted events
(i.e. `true_positive + false_positive = 0`).
Precision is undefined in this case, and those levels will be removed from the
averaged result.
Note that the following number of true events actually occurred for each
problematic event level:
'     ': 5
```

```
test_cm_glmnet <- test_pred_glmnet %>%
  conf_mat(truth = domain, estimate = .pred_class)

test_relaxed_glmnet <- relaxed_metrics(test_pred_glmnet)

relaxed_glmnet_cv
```

```
# A tibble: 1 x 2
  relaxed_accuracy     n
             <dbl> <int>
1            0.617   384
```

```
relaxed_svm_cv
```

```
# A tibble: 1 x 2
  relaxed_accuracy     n
             <dbl> <int>
1            0.701   384
```

And for the support vector machine:

```
final_fit_svm <- wf_svm %>% fit(data = train)
```

```
Warning: max_tokens was set to 8000, but only 3708 was available and selected.
max_tokens was set to 8000, but only 3708 was available and selected.
```

```
test_pred_svm <- predict(final_fit_svm, new_data = test) %>%
  bind_cols(test %>% select(domain, domain_secondary))

test_macro_f1_svm <- test_pred_svm %>%
  f_meas(truth = domain, estimate = .pred_class, estimator = "macro")

test_cm_svm <- test_pred_svm %>%
  conf_mat(truth = domain, estimate = .pred_class)

test_relaxed_svm <- relaxed_metrics(test_pred_svm)

test_macro_f1_svm
```

```
# A tibble: 1 x 3
  .metric .estimator .estimate
  <chr>   <chr>          <dbl>
1 f_meas  macro          0.703
```

```
test_cm_svm
```

```
                    Truth
Prediction
                      9                   0                   0
               2                   9                   1
               5                   0                  28
                  3                   4                   2
               0                   1                   2
                    Truth
```

```
Prediction
                              1                    0
                      1                    0
                      3                    1
                        22                    0
                      0                    4
```

```
test_relaxed_svm
```

```
# A tibble: 1 x 2
  relaxed_accuracy     n
             <dbl> <int>
1            0.796    98
```

The cross-validation results from the training stage show a relaxed accuracy of 0.701 for the SVM model and 0.617 for the logistic regression model. This indicates that the SVM classifier consistently outperforms the regularized logistic model during training.

When evaluated on the held-out test set, the SVM model achieves an even higher relaxed accuracy of 0.796, suggesting that the model generalizes well to unseen data. The consistency between cross-validation performance and test results indicates that the model is not strongly overfitting the training sample.

Taken together, these results demonstrate that machine learning models can effectively identify governance domains from policy implementation texts. The relatively high macro F1 score and relaxed accuracy suggest that the policy discourse exhibits detectable structural patterns that correspond to functional governance fields.

7. Policy Attention Measurement

After training the final classification model, the next step is to apply the model to the full corpus of policy paragraphs and construct a measure of policy attention across governance domains.

The goal of this stage is not only to classify individual paragraphs, but also to aggregate the predicted domain labels over time in order to observe how policy attention is distributed across different governance fields.

7.1 Final Model Training

Once the model selection process is completed, the final classifier is trained on the entire labeled dataset.

```
library(tidyverse)
library(tidymodels)
library(textrecipes)
library(tokenizers)
library(stringr)
library(lubridate)

project_dir <- path.expand("~/Desktop/chineseresearch")

out_dir   <- file.path(project_dir, "data", "07_attention")
model_dir <- file.path(project_dir, "data", "06_model")

dir.create(out_dir, showWarnings = FALSE, recursive = TRUE)
dir.create(model_dir, showWarnings = FALSE, recursive = TRUE)

out_pred <- file.path(out_dir, "para_predictions_class.csv")

rec_final <- recipe(
  domain ~ text + text_bigram + doc_type + topic_lvl1 +
    topic_lvl2 + dominant_topic + admin_score,
  data = label500
) %>%
  step_tokenize(text) %>%
  step_tokenfilter(text, max_tokens = 5000) %>%   #
  step_tfidf(text) %>%
  step_tokenize(text_bigram) %>%
  step_tokenfilter(text_bigram, max_tokens = 5000) %>%
  step_tfidf(text_bigram) %>%
  step_novel(all_nominal_predictors()) %>%
  step_unknown(all_nominal_predictors()) %>%
  step_dummy(all_nominal_predictors()) %>%
  step_normalize(admin_score)

svm_mod <- svm_linear(mode = "classification") %>%
  set_engine("LiblineaR")

wf_final <- workflow() %>%
  add_recipe(rec_final) %>%
```

```
  add_model(svm_mod)

set.seed(123)
final_fit <- fit(wf_final, data = label500)
```

At this stage, the training procedure no longer uses cross-validation because the goal is to obtain a single final model that incorporates all available labeled observations. The model is trained using the same preprocessing pipeline defined earlier, including TF-IDF features for unigrams and bigrams as well as structured policy metadata.

The trained workflow object is saved for later reuse.

```
saveRDS(final_fit, file.path(model_dir, "final_svm_workflow.rds"))
```

Saving the trained model allows the classification pipeline to be reused without retraining the model each time predictions are generated.

7.2 Batch Prediction of Policy Paragraphs

After training the final classifier, the model is applied to the full dataset of policy paragraphs. The dataset contains all paragraphs extracted from the policy documents under analysis.

```
para_master <- read_csv(para_file, show_col_types = FALSE) %>%
  mutate(
    paragraph_id = paste0(str_trim(url), "#", as.integer(para_id)),
    pubdate = as.Date(pubdate),
    year = year(pubdate),
    para_len = as.integer(para_len)
  ) %>%
  filter(!is.na(year))
```

This step loads the paragraph-level dataset and constructs several variables needed for later aggregation. Each paragraph is assigned a unique identifier (`paragraph_id`) based on the document URL and paragraph index. The publication date is converted into a year variable, which will later be used to track policy attention over time. Because the full dataset can be relatively large, predictions are performed in batches.

```
batch_size <- 1500
n <- nrow(para_master)
idx <- split(seq_len(n), ceiling(seq_len(n) / batch_size))
```

The dataset is divided into batches of 1500 observations to reduce memory usage and ensure that the prediction process remains stable even for large corpora.

For each batch, the paragraph text is preprocessed in the same way as the training data. In particular, bigram features are constructed and the necessary predictor variables are created.

```
for (b in seq_along(idx)) {
  message("Predict batch ", b, "/", length(idx))

  chunk <- para_master[idx[[b]], ] %>%
    transmute(
      paragraph_id,
      url,
      pubdate,
      year,
      para_len,
      text = str_squish(para_text),
      doc_type,
      topic_lvl1,
      topic_lvl2,
      dominant_topic = "0",
      admin_score = 0,
      text_bigram = map_chr(text, make_bigram_text)
    )

  pred_b <- predict(final_fit, new_data = chunk) %>%
    bind_cols(chunk %>% select(paragraph_id, url, pubdate, year, para_len)) %>%
    rename(pred_domain = .pred_class)

  if (!file.exists(out_pred)) {
    write_csv(pred_b, out_pred)
  } else {
    write_csv(pred_b, out_pred, append = TRUE)
  }

  rm(chunk, pred_b)
  gc()
}
```

These variables mirror the structure expected by the trained workflow, ensuring that the prediction pipeline is consistent with the original model training procedure. The classifier then generates predicted policy domains for each paragraph.

The predicted domain label (`pred_domain`) is stored together with the paragraph metadata. Each batch of predictions is appended to an output file. This step produces a dataset in which every paragraph in the policy corpus is assigned a predicted governance domain.

7.3 Constructing Policy Attention Measures

Once the full set of paragraph predictions is generated, the next step is to construct a measure of policy attention.

```
library(tidyverse)
library(lubridate)
library(scales)

project_dir <- "/Users/heyangliu/Desktop/chineseresearch"
pred_file <- file.path(project_dir, "data", "07_attention", "para_predictions_class.csv")

pred_all <- read_csv(pred_file, show_col_types = FALSE) %>%
  transmute(
    year = as.integer(year),
    pred_domain = str_squish(pred_domain),
    para_len = as.integer(para_len),
    weight = pmin(para_len, 300)
  ) %>%
  filter(
    !is.na(year),
    !is.na(pred_domain),
    pred_domain != ""
  )
```

This dataset contains the predicted policy domain for every paragraph in the corpus. In addition to the raw paragraph counts, I also construct a weighted measure based on paragraph length.

The variable `weight` assigns each paragraph a weight proportional to its length, capped at 300 characters. The intuition is that longer paragraphs typically contain more substantive policy discussion and therefore represent a stronger signal of policy attention.

The attention measure is then computed by aggregating paragraphs by year and predicted domain.

```r
att_fix <- pred_all %>%
  count(year, pred_domain, wt = NULL, name = "n") %>%
  left_join(
    pred_all %>%
      group_by(year, pred_domain) %>%
      summarise(w_sum = sum(weight, na.rm = TRUE), .groups = "drop"),
    by = c("year", "pred_domain")
  ) %>%
  group_by(year) %>%
  mutate(
    share = n / sum(n),
    w_share = w_sum / sum(w_sum)
  ) %>%
  ungroup() %>%
  arrange(year, pred_domain)

print(att_fix, n = 50)
```

```
# A tibble: 25 x 6
    year pred_domain            n   w_sum  share w_share
   <int> <chr>              <int>   <dbl>  <dbl>   <dbl>
 1  2021                8362  674330 0.135   0.113
 2  2021        8441  903663 0.137   0.151
 3  2021       27701 2519837 0.448   0.420
 4  2021          11602 1296078 0.188   0.216
 5  2021        5718  599183 0.0925  0.100
 6  2022                7785  713360 0.147   0.132
 7  2022        8706  988899 0.165   0.182
 8  2022       24084 2284922 0.456   0.421
 9  2022           7914  924767 0.150   0.171
10  2022        4354  511219 0.0824  0.0943
11  2023                4702  435323 0.120   0.109
12  2023        7640  865948 0.195   0.216
13  2023       14832 1422704 0.380   0.355
14  2023           8281  867985 0.212   0.217
15  2023        3627  416000 0.0928  0.104
16  2024                5197  452191 0.129   0.108
17  2024        7983  905042 0.198   0.216
18  2024       16988 1668768 0.422   0.398
19  2024           6666  720074 0.165   0.172
20  2024        3446  443294 0.0856  0.106
21  2025                2804  238172 0.0930  0.0736
```

```
22  2025        5820  666648 0.193   0.206
23  2025       14023 1484838 0.465   0.459
24  2025           4886  559401 0.162   0.173
25  2025        2617  286730 0.0868  0.0886
```

Two attention indicators are constructed:

1. Paragraph Share (share) The proportion of paragraphs in a given year that belong to each policy domain.
2. Weighted Attention (w_share) The proportion of weighted paragraph length assigned to each domain.

The first indicator captures how frequently each policy domain appears in the text, while the second accounts for the relative importance of longer policy paragraphs.

Together, these measures provide a time-series representation of policy attention allocation across governance domains.

8. Visualization of Policy Attention Structure

To illustrate how policy attention is distributed across governance domains, I construct several visualizations based on the aggregated attention measures. These figures display the yearly distribution of predicted policy domains between 2021 and 2025.

8.1 Trend of Policy Attention

The first visualization plots the yearly share of paragraphs belonging to each governance domain.

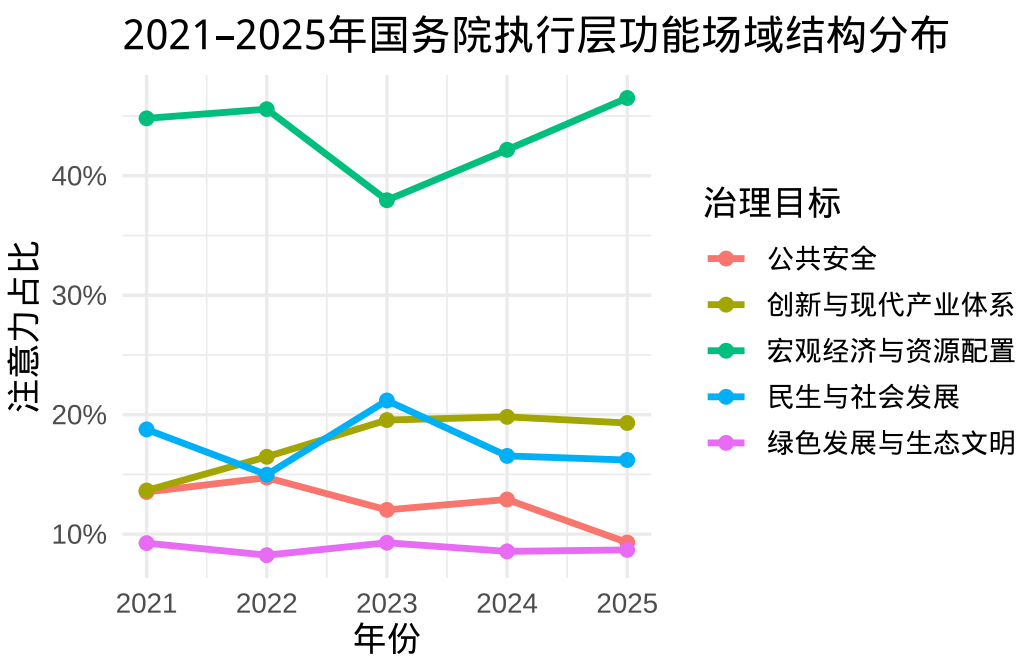
```r
library(tidyverse)

theme_set(
  theme_minimal(base_size = 13) +
    theme(
      text = element_text()
    )
)


p_trend <- att_fix %>%
  ggplot(aes(x = year, y = share, color = pred_domain, group = pred_domain)) +
```

```
  geom_line(linewidth = 1.2) +
  geom_point(size = 2) +
  scale_y_continuous(labels = percent_format(accuracy = 1)) +
  scale_x_continuous(breaks = 2021:2025) +
  labs(
    title = "2021-2025        ",
    x = " ",
    y = "   ",
    color = "   "
  )

p_trend
```



This line chart shows how the relative attention devoted to each governance domain evolves over time. Each line represents one policy domain, and the vertical axis measures the proportion of paragraphs classified into that domain within a given year.

The figure reveals that the distribution of policy attention is relatively stable across years, suggesting that the executive-level policy discourse maintains a consistent functional structure. However, some domains exhibit noticeable fluctuations, which may reflect shifting policy priorities or administrative campaigns.

In particular, the macroeconomic and resource allocation domain maintains a relatively high share of policy attention across the period. Social development policies also occupy a significant

portion of the policy discourse, while environmental governance and public security appear more episodically.

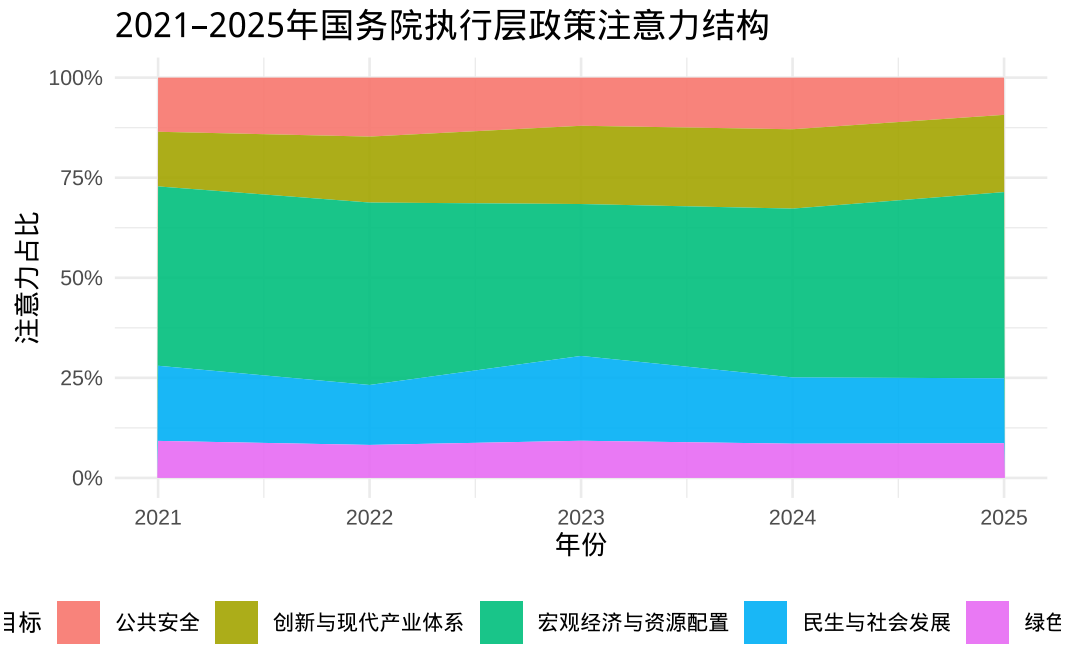
8.2 Structural Composition of Policy Attention

To visualize the overall composition of policy attention, I construct a stacked area chart.

```
theme_set(
  theme_minimal(base_size = 13) +
    theme(
      text = element_text(family = "PingFang SC"),
      legend.position = "bottom",
      panel.grid.minor = element_blank()
    )
)
p_area <- att_fix %>%
  ggplot(aes(x = year, y = share, fill = pred_domain)) +
  geom_area(position = "stack", alpha = 0.9) +
  scale_y_continuous(labels = scales::percent_format()) +
  labs(
    title = "2021-2025       ",
    x = " ",
    y = "   ",
    fill = "  "
  )  +
  theme_minimal(base_size = 10) +
  theme(
    text = element_text(family = "PingFang SC"),
    plot.title = element_text(size = 13, face = "bold"),
    strip.text = element_text(size = 10),
    axis.title = element_text(size = 10),
    axis.text = element_text(size = 8),
    legend.title = element_text(size = 9),
    legend.text = element_text(size = 8),
    legend.position = "bottom"
  )

p_area
```

```
Warning in grid.Call(C_stringMetric, as.graphicsAnnot(x$label)): font family
'PingFang SC' not found, will use 'sans' instead
```



In this figure, the vertical axis represents the total policy attention in each year, and the colored segments show the proportion allocated to each governance domain.

The stacked area visualization highlights the **functional composition of executive policy discourse**. Rather than focusing on the trajectory of individual domains, this representation emphasizes the relative balance among policy fields.

The figure shows that policy attention is distributed across multiple governance domains rather than concentrated in a single area. This pattern is consistent with the idea that executive-level policy implementation reflects a **multi-field governance structure**, where different administrative priorities coexist within the same policy system.

9. Discussion

9.1 Interpreting the Empirical Findings

This study set out to examine whether executive-level policy implementation texts exhibit a structured distribution of attention across governance domains. Using a supervised classification model trained on manually labeled paragraphs, the analysis assigns each policy paragraph to a governance field and aggregates these predictions into yearly attention shares.

The results provide several indications that policy discourse does exhibit a detectable functional structure. The classification model achieves a macro F1 score of approximately 0.70

on the held-out test set, and the relaxed accuracy metric reaches nearly 0.80. These results suggest that the vocabulary and linguistic patterns associated with different governance domains are sufficiently distinct for a machine learning classifier to identify them with reasonable reliability.

The attention aggregation results further show that policy attention is distributed across multiple governance domains rather than concentrated in a single field. Macroeconomic coordination and social development consistently account for a substantial portion of policy discourse, while other domains such as public security or environmental governance appear in more episodic patterns. These patterns are broadly consistent with existing literature on Chinese policy priorities, which highlights the central importance of economic management and social stability in administrative governance.

Taken together, the results support the broader interpretation that executive policy implementation exhibits a functional differentiation structure, in which multiple governance domains coexist and compete for policy attention within the same administrative system.

9.2 Limitations and Methodological Challenges

Despite the encouraging classification results, several methodological limitations remain and should be acknowledged.

First, the current training and validation strategy may introduce potential data leakage. In the current pipeline, paragraphs are randomly split into training and test sets. However, many paragraphs originate from the same policy document, which means that similar textual patterns may appear in both training and testing samples. This could artificially inflate model performance. A more rigorous strategy would involve grouped splitting and grouped cross-validation based on document identifiers, ensuring that paragraphs from the same document are not simultaneously used for both training and evaluation. Implementing this strategy would provide a more conservative and reliable estimate of the model's predictive performance.

Second, the distribution of labeled data across classes is highly uneven. Some governance domains contain only a small number of labeled observations. While having around thirty labeled instances for a class is not uncommon in exploratory machine learning projects, such a small sample size makes it difficult for the classifier to learn stable decision boundaries. Future work could address this issue in several ways. One approach would be to increase the size of the labeled dataset through additional manual annotation. Another possibility would be to collapse some closely related governance domains into broader categories, thereby increasing the effective sample size per class.

Third, the current classification framework assumes that each paragraph belongs to a single governance domain. In reality, however, policy texts frequently contain overlapping topics. A paragraph discussing digital infrastructure, for instance, may simultaneously relate to economic

development, technological innovation, and administrative regulation. Allowing multiple labels per paragraph could therefore better reflect the multidimensional nature of policy discourse. Methods such as Latent Dirichlet Allocation (LDA) or other topic modeling techniques were originally designed to capture such cross-domain membership. Future work could incorporate these approaches to model policy attention as a mixture of domains rather than a single categorical assignment.

Fourth, the current text processing pipeline may not fully capture the linguistic complexity of Chinese policy documents. While the present model relies on standard tokenization and bag-of-words representations, Chinese policy language often contains domain-specific terminology, multi-character expressions, and administrative formulae that may not be adequately captured by basic preprocessing techniques. More advanced approaches—such as custom domain dictionaries, phrase-level tokenization, or contextual embeddings from pretrained Chinese language models—could potentially improve classification performance.

Finally, the overall F1 score suggests that the amount of labeled data used for training remains limited. Although the current dataset is sufficient for building an initial prototype classifier, a larger labeled corpus would likely improve both predictive accuracy and model stability. Expanding the training dataset would also make it possible to experiment with more complex modeling approaches without the risk of overfitting.

9.3 Future Research: Machine Learning Improvements

While the current pipeline demonstrates that machine learning can be used to identify governance domains in policy texts, several methodological improvements could substantially strengthen the analysis.

9.3.1 Improving Data Splitting and Validation

A first priority is improving the evaluation strategy. In the current implementation, paragraphs are randomly split into training and testing sets. However, paragraphs originating from the same policy document often share vocabulary and stylistic patterns. As a result, paragraphs from the same document may appear in both training and testing samples, potentially leading to optimistic performance estimates.

Future versions of the pipeline should implement grouped splitting and grouped cross-validation based on document identifiers. Under this approach, all paragraphs belonging to the same document would be assigned to the same fold during cross-validation. This ensures that the model is evaluated on documents that were completely unseen during training, providing a more reliable estimate of generalization performance.

9.3.2 Expanding the Labeled Dataset

Another important improvement concerns the size of the labeled training dataset. The current model is trained on roughly 500 labeled paragraphs, which is sufficient for a prototype model but relatively small for a multi-class classification problem. The relatively modest F1 score suggests that the model is constrained by the amount of labeled data available for training.

Future work could expand the labeled dataset through additional manual annotation. Alternatively, active learning strategies could be implemented to prioritize the labeling of paragraphs that the model finds most uncertain. By iteratively selecting informative samples for annotation, active learning can significantly improve model performance while minimizing labeling costs.

9.3.3 Moving Toward Multi-Label Classification

The current classifier assumes that each paragraph belongs to exactly one governance domain. However, policy paragraphs often contain multiple thematic elements. For example, a paragraph discussing digital infrastructure investment could simultaneously relate to economic development, technological innovation, and environmental policy.

Future research could therefore adopt a multi-label classification framework, where paragraphs are allowed to belong to multiple domains simultaneously. This could be implemented using binary relevance classifiers or neural multi-label architectures.

Another possibility would be to use topic models such as Latent Dirichlet Allocation (LDA) to represent each paragraph as a distribution over multiple governance domains. Rather than assigning a single label, this approach would estimate the proportion of each domain present within the text.

9.3.4 Improving Chinese Text Representation

The current pipeline uses a relatively simple bag-of-words representation with TF-IDF weighting. While this approach is computationally efficient, it may not fully capture the semantic structure of Chinese policy language.

Future improvements could involve more advanced text representations, such as:

- pretrained Chinese BERT embeddings
- contextual sentence embeddings
- phrase-level tokenization using domain dictionaries

These methods allow the model to capture semantic relationships between words and phrases, which may improve classification performance for policy texts that rely heavily on administrative terminology.

9.3.5 Model Interpretability and Feature Attribution

Another direction for future research involves improving model interpretability. Although the current classifier provides domain predictions, it does not directly reveal which textual features drive these predictions.

Techniques such as Shapley value decomposition (SHAP) could be applied to the trained model to identify which tokens or features contribute most strongly to each classification decision. This would allow researchers to examine how specific policy terms influence domain predictions and would provide a more transparent connection between linguistic features and governance domains.